

数据结构课程设计实验报告

第 20 小组

韩旭成 胡宇杰 王跃然

2026 年 6 月 16 日

实习 2.8 电梯模拟

实习 4.1 稀疏矩阵运算器

目录

1	电梯模拟	3
1.1	问题描述	3
1.2	数据结构设计	3
1.2.1	乘客	3
1.2.2	电梯	3
1.2.3	事件	4
1.3	算法设计与优化	5
1.3.1	事件维护	5
1.3.2	乘客行为模拟	6
1.3.3	乘客生成策略	8
1.4	程序运行效果	9
1.5	扩展：多电梯模拟系统	10
1.6	总结	15
2	稀疏矩阵运算器	15
2.1	问题描述	15
2.2	数据结构设计	15
2.3	算法设计与优化	16
2.3.1	十字链表的创建	16
2.3.2	稀疏矩阵的加减	17
2.3.3	稀疏矩阵相乘	19
2.3.4	主函数	21
2.4	程序运行效果	21
2.5	总结	22
3	源代码	22

1 电梯模拟

1.1 问题描述

设计一个离散的模拟程序模拟电梯系统。在该程序中，以模拟时钟决定每个活动体（电梯、乘客）的动作发生的时刻和顺序，系统在某个模拟瞬间处理有待完成的各种事情，然后把模拟时钟推进到某个动作预定要发生的下一个时刻。

在简单模拟中，假设电梯运行范围为 5 层，且只有一部电梯，乘客随机出现在某一楼层，每名乘客有忍耐时间如果等待时间超过了忍耐时间乘客就会放弃乘梯并离开。电梯应尽量运送尽可能多的乘客，如果没有乘客，电梯应在最后停留的楼层等待一段时间后返回一楼。

在简单模拟的基础上，增加电梯运行范围与数量，研究多梯运行的最佳策略。

1.2 数据结构设计

单电梯模拟任务中，有关数据结构定义如下。

1.2.1 乘客

对于乘客，除了存储必要数据（编号、进出楼层、容忍时间、下一乘客出现时间）以外，还需要存储乘客的状态（等待还是已经进入电梯）。

```
1 typedef struct Passenger {
2     int id;
3     int in_floor;
4     int out_floor;
5     int arrive_time;
6     int giveup_time;
7     int entered;           // 乘客是否进入电梯
8     int assigned_elev;    // 对于多梯模拟，存储欲分配的电梯编号
9 } Passenger;
```

1.2.2 电梯

对于电梯，需要存储电梯中的乘客信息、电梯各按钮的状态（包括梯内楼层按钮和梯外乘客按动的呼叫按钮）、当前电梯外各层的排队情况、电梯的运行状态、停留时间上限

```
1 // 电梯内的乘客存储栈，遵循后进先出
2 typedef struct {
3     Passenger* stack[CAPACITY];
4     int top;
```

```
5 } ElevatorStack;
6
7 //等待队列，采用循环队列结构，遵循先进先出
8 typedef struct {
9     Passenger* queue[100];
10    int front;
11    int rear;
12 } WaitQueue;
13
14 //电梯数据结构
15 typedef struct {
16    int id;
17    int floor;
18    int state;
19    int door_state;
20    int call_up[FLOORS];
21    int call_down[FLOORS];
22    int call_car[FLOORS];
23    ElevatorStack passengers;
24    int moving; //电梯是否在移动的标识
25    int idle_start_time;
26    WaitQueue wait_queues[FLOORS];
27 } Elevator;
```

1.2.3 事件

将所有事件分为几类（如乘客进出事件、放弃时间、电梯开关门事件等），存储事件的类别、发生时间、必要数据等。使用堆来管理所有未来事件（到达、移动、开门、关门、进出、放弃、空闲超时），按时间排序，堆顶为最早事件。

```
1 //事件类型的枚举结构定义
2 typedef enum {
3     EV_PASSENGER_ARRIVE,
4     EV_ELEV_ARRIVE,
5     EV_DOOR_OPEN,
6     EV_PERSON_EXIT,
7     EV_PERSON_ENTER,
8     EV_DOOR_CLOSE,
9     EV_GIVEUP,
10    EV_IDLE_BACK
```

```

11 } EventType;
12
13 typedef struct {
14     int time;
15     EventType type;
16     int elev_id;
17     union {
18         struct { Passenger* p; } giveup;
19         struct { int floor; } elev_arrive;
20     } data; //事件的关键信息，包括乘客放弃事件中的乘客结点电梯到达事
              件的到达楼层
21 } Event;

```

1.3 算法设计与优化

1.3.1 事件维护

我们使用离散事件驱动模拟时钟，模拟时钟跳跃式推进。并使用最小堆管理事件，将所有未来事件（到达、移动、开门、关门、进出、放弃、空闲超时）按时间排序，其中堆顶为最早事件。每次处理堆顶事件可以避免时间轮询，效率高。

最小堆操作中，插入和取出事件的时间复杂度均为 $O(\log n)$ ，且可以支持大量动态事件。关键代码如下。

```

1  /* ----- 堆操作 ----- */
2  void heap_push(Event e) {
3      if (heap_size >= heap_capacity) {
4          heap_capacity = heap_capacity ? heap_capacity * 2 : 1000;
5          event_heap = (Event*)realloc(event_heap, heap_capacity *
              sizeof(Event));
6      }
7      int i = heap_size++;
8      while (i > 0) {
9          int parent = (i - 1) / 2;
10         if (event_heap[parent].time <= e.time) break;
11         event_heap[i] = event_heap[parent];
12         i = parent;
13     }
14     event_heap[i] = e;
15 }
16

```

```
17 Event heap_pop(void) {
18     Event top = event_heap[0];
19     Event last = event_heap[--heap_size];
20     int i = 0;
21     while (i * 2 + 1 < heap_size) {
22         int child = i * 2 + 1;
23         if (child + 1 < heap_size && event_heap[child + 1].time <
24             event_heap[child].time)
25             child++;
26         if (last.time <= event_heap[child].time) break;
27         event_heap[i] = event_heap[child];
28         i = child;
29     }
30     event_heap[i] = last;
31     return top;
}
```

1.3.2 乘客行为模拟

将栈与队列的混合应用，电梯内用栈（后进先出）模拟乘客挤在门口先出的行为；每层等候区用队列（先进先出）保证先到先服务。

```
1 void person_exit(void) {
2     int found = -1;
3     for (int i = elevator.passengers.top; i >= 0; i--) {
4         if (elevator.passengers.stack[i]->out_floor == elevator.floor
5             ) {
6             found = i;
7             break;
8         }
9     }
10    if (found >= 0) { //栈弹出操作
11        Passenger* p = elevator.passengers.stack[found];
12        for (int i = found; i < elevator.passengers.top; i++)
13            elevator.passengers.stack[i] = elevator.passengers.stack[
14                i + 1];
15        elevator.passengers.top--;
16        completed_passengers++;
17        printf("[%6d] Passenger %d exits elevator (target floor %d)\n",
18            current_time, p->id, p->out_floor);
19    }
```

```
16     free(p);
17
18     int still_has = 0; //找寻还有没有在当前层没下电梯的乘客并规划
    下一事件
19     for (int i = 0; i <= elevator.passengers.top; i++) {
20         if (elevator.passengers.stack[i]->out_floor == elevator.
            floor) {
21             still_has = 1;
22             break;
23         }
24     }
25     if (still_has)
26         schedule_event(EXIT_TIME, EV_PERSON_EXIT, NULL);
27     else if (wait_queues[elevator.floor].front != wait_queues[
        elevator.floor].rear)
28         schedule_event(ENTER_TIME, EV_PERSON_ENTER, NULL);
29     else
30         schedule_event(DOOR_CLOSE, EV_DOOR_CLOSE, NULL);
31 }
32 }
33
34 void person_enter(void) {
35     WaitQueue* wq = &wait_queues[elevator.floor];
36     if (wq->front == wq->rear) { //当前层无人
37         schedule_event(DOOR_CLOSE, EV_DOOR_CLOSE, NULL);
38         return;
39     }
40     if (elevator.passengers.top >= CAPACITY - 1) { //电梯已满
41         schedule_event(DOOR_CLOSE, EV_DOOR_CLOSE, NULL);
42         return;
43     }
44     Passenger* p = wq->queue[wq->front++];
45     if (wq->front >= 100) wq->front = 0;
46     if (p->entered) {
47         return;
48     }
49     p->entered = 1; //进入电梯
50     elevator.passengers.stack[++elevator.passengers.top] = p;
51     elevator.call_car[p->out_floor] = 1;
52     printf("[%6d] Passenger %d enters elevator, target floor %d\n",
```

```

        current_time, p->id, p->out_floor);
53 //规划下一事件（关门时间或乘客进入时间）
54 if (wq->front != wq->rear && elevator.passengers.top < CAPACITY -
    1)
55     schedule_event(ENTER_TIME, EV_PERSON_ENTER, NULL);
56 else
57     schedule_event(DOOR_CLOSE, EV_DOOR_CLOSE, NULL);
58 }

```

1.3.3 乘客生成策略

起终点按概率偏向 1 层（大厅），容忍等待时间与楼层间距成正比，随机波动，以模拟真实行为。

```

1 int generate_giveup_time(int in, int out) { //容忍时间生成：与楼层间
    距正相关
2     int span = abs(in - out);
3     int base = (in > out) ? DOWNToleranceBase : UPToleranceBase;
4     int extra = rand() % (span * 20 + 1);
5     int time = base * span + extra;
6     if (time < 20) time = 20;
7     return time;
8 }
9
10 int generate_inter_time(void) {
11     return random_range(MIN_INTER_TIME, MAX_INTER_TIME);
12 }
13
14 Passenger* create_passenger(void) {
15     static int next_id = 0;
16     Passenger* p = (Passenger*)malloc(sizeof(Passenger));
17     p->id = next_id++;
18     p->entered = 0;
19
20     double r = (double)rand() / RAND_MAX;
21     if (r < PROB_FLOOR1 / 2) { //有PROB_FLOOR1（本实验设置为0.5）的概率，
        乘客的出发地或目的地与一楼有关
22         p->in_floor = 1;
23         do { p->out_floor = rand() % FLOORS; } while (p->out_floor ==
            1);

```



```
24     }
25     else if (r < PROB_FLOOR1) {
26         p->out_floor = 1;
27         do { p->in_floor = rand() % FLOORS; } while (p->in_floor ==
28             1);
29     }
30     else {
31         do {
32             p->in_floor = rand() % FLOORS;
33             p->out_floor = rand() % FLOORS;
34         } while (p->in_floor == p->out_floor);
35     }
36     p->arrive_time = current_time;
37     p->giveup_time = current_time + generate_giveup_time(p->in_floor,
38         p->out_floor);
39     return p;
}
```

1.4 程序运行效果

单电梯模拟运行结果如下：

```
PS E:\code\datastructure\prj\simulation> gcc -o elevator main.c elevator_core.c
PS E:\code\datastructure\prj\simulation> ./elevator
[ 80] Passenger 0 arrives at floor 1 -> floor 3, tolerance = 1003
[ 100] Elevator door opens
[ 125] Passenger 0 enters elevator, target floor 3
[ 145] Elevator door closes
[ 145] Elevator moving to floor 3
[ 191] Passenger 1 arrives at floor 0 -> floor 1, tolerance = 505
[ 196] Elevator arrives at floor 3
[ 216] Elevator door opens
[ 241] Passenger 0 exits elevator (target floor 3)
[ 261] Elevator door closes
[ 292] Passenger 2 arrives at floor 3 -> floor 1, tolerance = 812
[ 312] Elevator door opens
[ 314] Passenger 3 arrives at floor 1 -> floor 4, tolerance = 1504
[ 337] Passenger 2 enters elevator, target floor 1
[ 357] Elevator door closes
[ 357] Elevator moving to floor 1
[ 418] Elevator arrives at floor 1
[ 438] Elevator door opens
[ 463] Passenger 2 exits elevator (target floor 1)
[ 477] Passenger 4 arrives at floor 4 -> floor 2, tolerance = 834
[ 488] Passenger 3 enters elevator, target floor 4
[ 508] Elevator door closes
[ 508] Elevator moving to floor 4
[ 559] Elevator arrives at floor 4
[ 579] Elevator door opens
[ 604] Passenger 3 exits elevator (target floor 4)
[ 629] Passenger 4 enters elevator, target floor 2
[ 649] Elevator door closes
[ 649] Elevator moving to floor 2
[ 673] Passenger 5 arrives at floor 1 -> floor 3, tolerance = 1010
[ 696] Passenger 1 gives up waiting (timeout)
```

图 1: 单电梯模拟运行效果

```
==== Simulation finished =====
Total passengers: 58
Completed: 29
Give up: 24
Completion rate: 50.0%
Simulation time: 501.0 seconds
```

图 2: 单电梯模拟运行结果

从图 1 的运行过程中可以看到乘客到达、进出电梯、放弃以及电梯开关门和移动等事件。以乘客 1 为例，他于 191 时刻进入系统，容忍时间 505 时间单位，因此在 696 时刻在没有等到电梯的情况下放弃并离开系统。图 2 为这次模拟的最终数据统计，可以看到成功率是比较高的。

1.5 扩展：多电梯模拟系统

利用数组 `elevators[]` 存储多部电梯，每部电梯拥有独立的状态、按钮、乘客栈和等待队列。每部电梯有自己的事件链（移动、开门、关门等），事件堆中混排所有电梯的事

件，但通过 elev_id 区分。运行时通过命令行参数指定电梯数量（例如./elevator 4），可支持 1 到 10 部电梯。从单电梯到多电梯只需增加调度器函数和电梯数组，核心动作函数（open_door、person_enter 等）通过 elev_id 参数复用，扩展性强。

在单电梯的基础上，多电梯涉及到调度策略。使用函数 dispatch_elevator() 为每个新乘客遍历所有电梯，综合距离、方向、负载、门状态等计算评分，返回最优电梯 ID。

具体而言，对每部电梯计算一个“代价分数”，选分数最低者。评分因素包括：1. 电梯与请求楼层的绝对距离 2. 若电梯方向与请求方向一致且顺路时，减分 3. 若电梯空闲，大幅减分（优先响应）4. 电梯当前乘客数（负载均衡，避免某梯过挤）。算法如下。

```
1 int dispatch_elevator(int floor, int direction) {
2     int best_id = -1;
3     int best_score = 999999;
4
5     for (int i = 0; i < num_elevators; i++) {
6         Elevator* e = &elevators[i];
7         int score = 0;
8         int dist = abs(e->floor - floor);
9         score += dist * 10;
10        if (e->state == GOING_UP && direction == DIR_UP && e->floor
11            <= floor)
12            score -= 30;
13        else if (e->state == GOING_DOWN && direction == DIR_DOWN && e
14            ->floor >= floor)
15            score -= 30;
16        if (e->state == IDLE)
17            score -= 100;
18        if (e->door_state == DOOR_OPENED)
19            score += 200;
20        score += e->passengers.top * 5;
21
22        if (score < best_score) {
23            best_score = score;
24            best_id = i;
25        }
26    }
27    return best_id;
28 }
```

多电梯模拟系统的运行结果如下。

```
PS E:\code\datastructure\prj\multi> ./elevator 3
[ 55] Passenger 0 arrives at floor 0 -> floor 1, tolerance = 218, assigned to elevator 0
[ 239] Passenger 1 arrives at floor 3 -> floor 1, tolerance = 323, assigned to elevator 0
[ 255] Passenger 2 arrives at floor 1 -> floor 0, tolerance = 166, assigned to elevator 0
[ 273] Passenger 0 gives up waiting (timeout) for elevator 0
[ 275] Elevator 0 door opens
[ 300] Passenger 2 enters elevator 0, target floor 0
[ 318] Passenger 3 arrives at floor 0 -> floor 1, tolerance = 212, assigned to elevator 1
[ 320] Elevator 0 door closes
[ 320] Elevator 0 moving to floor 0
[ 381] Elevator 0 arrives at floor 0
[ 401] Elevator 0 door opens
[ 426] Passenger 2 exits elevator 0 (target floor 0)
[ 446] Elevator 0 door closes
[ 454] Passenger 4 arrives at floor 0 -> floor 4, tolerance = 856, assigned to elevator 0
[ 474] Elevator 0 door opens
[ 499] Passenger 4 enters elevator 0, target floor 4
[ 519] Elevator 0 door closes
[ 519] Elevator 0 moving to floor 4
[ 530] Passenger 3 gives up waiting (timeout) for elevator 1
[ 536] Passenger 5 arrives at floor 4 -> floor 1, tolerance = 485, assigned to elevator 1
[ 562] Passenger 1 gives up waiting (timeout) for elevator 0
```

图 3: 多电梯模拟运行效果

```
===== Multi-Elevator Simulation Finished =====
Number of elevators: 3
Total passengers: 52
Completed: 30
Give up: 19
Completion rate: 57.7%
Simulation time: 502.5 seconds
```

图 4: 多电梯模拟运行结果

可以看到，不同乘客被分配到了不同电梯。

上述代码中的数字是权重，可以被训练，为了尝试优化调度策略，我们设计了一个简单的基于随机爬山法的参数优化算法。采用同样的十组随机种子进行训练，每次训练时在当前权重附近随机取值（即“爬山”）并计算新的损失函数，寻找使损失函数最小的权重值。

损失函数采用乘客等待时间平方之和，以惩罚较长的等待时间。如果乘客放弃，进行额外惩罚；如果乘客成功进入电梯，进行额外奖励。

```
1 for (int i = 0; i < 2000; i++) {
2     DispatchWeights *new_w = generate_w(&best_w, stride);
3     W = *new_w;
4     total = 0;
5     com = 0;
6     giveup = 0;
7     evaluate_weights(elev_count);
8     if (loss < best_loss) {
9         times = i;
10        best_loss = loss;
11        best_w = *new_w;
```

```
12     best_total = total;
13     best_com = com;
14     best_giveup = giveup;
15 }
16 if (i == times + 500) { //连续500次没有更小损失出现就结束
17     printf("Training\times:\t%d\n", times);
18     break;
19 }
20 if (i % 100 == 0 && i > 0) {
21     stride *= 0.8;
22 }
23 if (i % 50 == 0) {
24     printf("current\tweights:\t%.6f\t%.6f\t%.6f\t%.6f\t%.6f\n", best_w.
25         w_dist, best_w.w_same_dir, best_w.w_idle, best_w.
26         w_door_open, best_w.w_load);
27     printf("current\tloss:\t%.6f\n", best_loss);
28 }
29 }
30 void evaluate_weights(int elev_count)
31 {
32     loss = 0;
33     for (int j = 0; j < 10; j++) { //跑十组训练数据
34         total_passengers = completed_passengers = giveup_passengers =
35             0;
36         current_time = 0;
37         init_system(elev_count);
38         set = j;
39         next_id = 0;
40         schedule_event(p_data[set][0].appear_time,
41             EV_PASSENGER_ARRIVE, -1, NULL);
42         while (!heap_empty() && current_time < SIM_TIME) {
43             Event e = heap_pop();
44             current_time = e.time;
45             process_event(&e);
46         }
47         total += total_passengers;
48         com += completed_passengers;
49         giveup += giveup_passengers;
50     }
51 }
```

```

48 DispatchWeights *generate_w(DispatchWeights *pre_w, double stride) //
    随机生成下一组权重值
49 {
50     DispatchWeights *new_w = (DispatchWeights *) malloc(sizeof(
        DispatchWeights));
51     new_w->w_dist = pre_w->w_dist + ((double) rand() / RAND_MAX * 2.0
        - 1.0) * stride;
52     new_w->w_door_open = pre_w->w_door_open + ((double) rand() /
        RAND_MAX * 2.0 - 1.0) * stride;
53     new_w->w_idle = pre_w->w_idle + ((double) rand() / RAND_MAX * 2.0
        - 1.0) * stride;
54     new_w->w_load = pre_w->w_load + ((double) rand() / RAND_MAX * 2.0
        - 1.0) * stride;
55     new_w->w_same_dir = pre_w->w_same_dir + ((double) rand() /
        RAND_MAX * 2.0 - 1.0) * stride;
56     return new_w;
57 }
58 // 损失函数
59 loss += (current_time - p->arrive_time) * (current_time - p->
    arrive_time) / 100.0; // 乘客进入电梯
60 loss -= 5000; // 乘客出电梯时进行奖励
61 loss += (current_time - p->arrive_time + 500) * (current_time - p->
    arrive_time + 500) / 100.0; // 乘客放弃时进行惩罚

```

检验我们的算法能否优化调度策略，提高电梯接人成功率。记录数据如下（不同序号对应不同的训练种子）。

表 1: 多电梯系统调度策略优化算法数据记录表（电梯数为 3，损失单位 10^4 ）

序号	初始成功率	初始权重 w_dist	初始损失	最终成功率	最终权重 w_dist	最终损失
1	53.8%	10.00	7.63	56.2%	10.26	-8.86
2	54.8%	10.00	5.95	54.8%	15.43	-0.37
3	57.8%	10.00	-9.62	58.6%	11.55	-24.90
4	57.1%	10.00	-1.78	58.1%	12.17	-14.16
5	55.6%	10.00	9.87	58.0%	14.15	-5.13
6	54.1%	10.00	27.40	52.7%	7.09	1.35
7	57.1%	10.00	-16.58	59.9%	7.95	-24.37
8	54.6%	10.00	12.74	55.4%	15.40	-11.08
9	56.0%	10.00	-10.43	58.7%	7.51	-15.79
10	56.2%	10.00	-2.89	58.2%	14.25	-7.47

总结实验结果如下：

1. 在大部分情况下，训练后得到的最终权重成功率比初始成功率高 1 到 2 个百分点，损失函数大幅下降，说明该优化算法是有效的。
2. 对于不同数据，训练出的最终权重差异很大，说明在随机乘客模拟的情况下，难以找到一个最优权重作为“通解”。

实际应用中，因为乘客出现时间是不能预知的，所以该算法不能用来预测真实情况下的权重。但如果加以限制条件（如高峰期，乘客出现的规律满足某一正态分布或泊松分布），这样数据会有较高的一致性，采用大量这类型的数据训练权重，并再次优化算法设计，可能可以得到某个次优通解

1.6 总结

本实验中，我们完成了单电梯、多电梯模拟两个任务，经检验程序可以正常运行，效率较高。

未来改进方向：优化权重学习算法，增加 GUI 图形界面等。

2 稀疏矩阵运算器

2.1 问题描述

本实验旨在设计并实现一个简单的稀疏矩阵运算器，以三元组形式输入稀疏矩阵的数据，实现多个稀疏矩阵的加法、减法与乘法运算，并将运算结果以阵列的形式输出。

在保障运算器功能正确的前提下，应尽量提升运算器的时间、空间效率。

2.2 数据结构设计

我们采用十字链表存储矩阵，并实现了十字链表存储矩阵的加法、减法、乘法操作。十字链表需要定义结点与行列头指针，还需要定义矩阵有关的数据结构存储矩阵。具体定义如下。

```
1 //数据结构：十字链表
2 typedef struct OLNode
3 {
4     int i, j; //行列值
5     int e; //元素值
6     OLNode* right, * down; //指向右侧、下方的下一个非零元
7 }OLNode;
8
```

```

9 typedef struct
10 {
11     vector<OLNode*> rhead, chead; // 储存每一行、每一列第一个非零元的指针
12     int mu, nu, tu; // 矩阵的行数、列数、非零元个数
13 }Crosslist;

```

2.3 算法设计与优化

以十字链表结构为基础，稀疏矩阵运算器主要实现了以下函数。

```

1 //根据以行序输入的数据创建十字链表
2 void CreateOLMatrix(Crosslist* M);
3
4 //基于十字链表的加减、乘法，结果以十字链表存储
5 void AddMinusOLMatrix(Crosslist* A, Crosslist* B, int op);
6 void MultiOLMatrix(Crosslist* A, Crosslist* B, Crosslist* M);
7
8 //打印结果和释放节点
9 void PrintOLMatrix(Crosslist* M);
10 void FreeCrosslist(Crosslist* M);

```

2.3.1 十字链表的创建

对于十字链表创建函数 CreateOLMatrix，因为要求以行序输入，因此每一次插入节点时不需要寻找位置。只需要记录当前行和每一列的前驱，每次在前驱后加入即可。将插入的复杂度降至 $O(t)$ 。列的逻辑类似，但是需要数组维护每一列的前驱。以行序为例展示插入逻辑。

```

1 int row = 0;
2 OLNode* currentRow = NULL; // 行的当前指针
3 for (int k = 0; k < M->tu; k++){
4     if (icur != row){
5         row = icur;
6         M->rhead[icur] = p; // 换行后更新rhead
7     }
8     else
9         currentRow->right = p;
10    currentRow = p; // 更新行指针
11 }

```


2.3.2 稀疏矩阵的加减

稀疏矩阵加减函数 AddMinusOLMatrix 均采用一般的方法，逐行扫描增删节点，并根据 pa,pb 指针的位置分情况讨论。

如果 pa,pb 对应位置一致，那么当前节点的值更新为两者的和或差，如果和或差为 0，就要删去该节点。如果 pa 在 pb 之前，pa 保持不动。如果 pb 在 pa 之前，就增加一个节点，其值与 pb 对应节点一致。时间复杂度是 $O(ta + tb)$ 。

```

1 void AddMinusOLMatrix(Crosslist* A, Crosslist* B, int op)
2 {
3     vector<OLNode*> h1(A->nu + 1, NULL);
4     for (int col = 1; col <= A->nu; col++)
5         h1[col] = A->chead[col];
6     //辅助指针h1, 类似与行的前驱pre
7     for (int row = 1; row <= A->mu; row++)
8     {
9         OLNNode* pa = A->rhead[row], * pb = B->rhead[row], * pre =
            NULL; //辅助指针pre
10        while (pb != NULL)
11        {
12            if (pa == NULL || pa->j > pb->j) //表示A中要新增一个来自B
                的结点
13            {
14                OLNNode* p = new OLNNode;
15                p->right = NULL; p->down = NULL;
16                p->i = pb->i;
17                p->j = pb->j;
18                if (op == 1)
19                    p->e = pb->e;
20                else
21                    p->e = -pb->e;
22                if (pre == NULL) //p将成为本行第一个非零元
23                    A->rhead[row] = p;
24                else
25                    pre->right = p;
26                p->right = pa;
27                pre = p;
28                if (!A->chead[p->j] || A->chead[p->j]->i > p->i)
29                {
30                    //p将成为本列第一个非零元, chead也要随之更改
31                    p->down = A->chead[p->j];

```

```
32         A->chead[p->j] = p;
33     }
34     else
35     {
36         //在h1的基础上再向下找
37         while (h1[p->j]->down && h1[p->j]->down->i < p->i
38             )
39             h1[p->j] = h1[p->j]->down;
40         p->down = h1[p->j]->down;
41         h1[p->j]->down = p;
42     }
43     h1[p->j] = p;
44     pb = pb->right;
45 }
46 else if (pa->j < pb->j)
47 {
48     //沿用A中的结点，但是要更新pre和pa
49     pre = pa;
50     pa = pa->right;
51 }
52 else
53 {
54     //此时A、B在此位置均有非零元，要进行计算
55     if (op == 1)
56         pa->e += pb->e;
57     else
58         pa->e -= pb->e;
59     pb = pb->right;
60     if (pa->e == 0)
61     {
62         if (pre == NULL)//删除的是本行原来的第一个非零元
63             A->rhead[row] = pa->right;
64         else
65             pre->right = pa->right;
66         OLNode* p = pa;
67         pa = pa->right;
68         if (A->chead[p->j] == p)
69         {
70             //删除的是本列第一个非零元
71             A->chead[p->j] = p->down;
```

```

71         hl[p->j] = p->down;
72     }
73     else
74     {
75         //通过hl向下找到被删节点的前驱
76         while (hl[p->j]->down && hl[p->j]->down->i <
77             p->i)
78             hl[p->j] = hl[p->j]->down;
79         hl[p->j]->down = p->down;
80     }
81     delete p;
82 }
83 else
84 {
85     pre = pa; //重要, 记得更新
86     pa = pa->right;
87 }
88 }

```

2.3.3 稀疏矩阵相乘

与行逻辑链接的顺序表思路类似, 逐行求积, 每次 A 中一个元素乘 B 的一行, 计算它对于这一行的贡献。每完成一行的计算之后, 类似于 CreateOLMatrix 插入节点。

```

1 void MultiOLMatrix(Crosslist* A, Crosslist* B, Crosslist* M)
2 {
3     //根据A,B的行列值对M的行列赋值
4     M->mu = A->mu; M->nu = B->nu; M->tu = 0;
5     M->rhead.resize(M->mu + 1, NULL);
6     M->thead.resize(M->nu + 1, NULL);
7     int row = 0;
8     OLNNode* currentRow = NULL;
9     vector<OLNNode*> currentCol(M->nu + 1, NULL);
10    //每次通过A中的一行, 算出M中对应行的值
11    for (int i = 1; i <= M->mu; i++)
12    {
13        vector<int> ctemp(M->nu + 1, 0);
14        OLNNode* Atemp = A->rhead[i];
15        while (Atemp)

```

```
16     {
17         int Adata = Atemp->e;
18         OLNode* Btemp = B->rhead[Atemp->j];
19         while (Btemp) {
20             int Bdata = Btemp->e;
21             ctemp[Btemp->j] += Adata * Bdata;
22             Btemp = Btemp->right;
23         }
24         Atemp = Atemp->right;
25     }
26     //每一行计算完毕之后在十字链表中创建节点, 与CreateOLMatrix()
    类似
27     for (int k = 1; k <= M->nu; k++)
28     {
29         if (ctemp[k])
30         {
31             M->tu++;
32             int icur, jcur, ecur;
33             icur = i;    jcur = k;    ecur = ctemp[k];
34             OLNode* p = new OLNode;
35             p->i = icur; p->j = jcur; p->e = ecur;
36             p->down = NULL; p->right = NULL;
37             if (icur != row) {
38                 row = icur;
39                 M->rhead[icur] = p;
40             }
41             else {
42                 currentRow->right = p;
43             }
44             currentRow = p;
45             if (currentCol[jcur] == NULL) {
46                 M->thead[jcur] = p;
47             }
48             else {
49                 currentCol[jcur]->down = p;
50             }
51             currentCol[jcur] = p;
52         }
53     }
```

我们同样也实现了以行逻辑链接的顺序表存储稀疏矩阵的矩阵乘法，具体可见源码。

2.3.4 主函数

在主函数中，先输入一个矩阵，之后每次输入一个运算类型和一个矩阵。得到输入后先判断大小是否合法，如果合法则输出这一步运算的结果，并且将该结果作为下一步运算的数据；否则要求重新输入。整体结构如下。

```
1 while (1) {
2     cin >> Src->mu >> Src->nu >> Src->tu;
3     CreateOLMatrix(Src);
4     while (1) {
5         cin >> op;    cin >> Tar->mu >> Tar->nu >> Tar->tu;
6         if (op == 1 || op == 2) //加减
7         else if (op == 3) //乘法
8         else if (op == 0) break; //退出当前计算
9         else cout << "操作类型不合法!" << endl;
10    }
11 }
12 //加减法判断分支的架构
13 if (op == 1 || op == 2){
14     if (Tar->mu != Src->mu || Tar->nu != Src->nu) {
15         delete Tar;
16         continue;
17     } //对数据进行检查
18     CreateOLMatrix(Tar); //符合要求才能进一步输入数据
19     AddMinusOLMatrix(Src, Tar, op); //结果储存在Src中，以便后续使用
20     PrintOLMatrix(Src); //打印当前结果
21     FreeCrosslist(Tar); // Tar后续不再使用，需要清除
22 }
```

2.4 程序运行效果

我们设计的稀疏矩阵运算器经检验，能进行矩阵的加减乘三种运算操作；能简单地判断输入矩阵的大小是否合法；能够顺序地实现连续运算。

```
请输入第1个矩阵的行数、列数、非零元数：
3 4 1
以三元组形式输入第1个矩阵的数据(行序，从1开始)：
1 1 1
请输入运算类型（1-加；2-减；3-乘；0-退出运算）：
1
请输入第2个矩阵的行数、列数、非零元数：
4 3 2
这组数据不符合要求，请重新输入。 请输入运算类型（1-加；

请输入运算类型（1-加；2-减；3-乘；0-退出运算）：
4
操作类型不合法！
请输入运算类型（1-加；2-减；3-乘；0-退出运算）：
```

对于错误输入的判定

```
请输入第1个矩阵的行数、列数、非零元数：
3 4 3
以三元组形式输入第1个矩阵的数据(行序，从1开始)：
1 2 1
1 3 2
2 1 6
请输入运算类型（1-加；2-减；3-乘；0-退出运算）：
3
请输入第2个矩阵的行数、列数、非零元数：
4 3 4
以三元组形式输入第2个矩阵的数据(行序，从1开始)：
1 1 1
2 2 1
4 1 2
4 3 4
计算结果如下：
0 1 0
6 0 0
0 0 0
A×B

请输入运算类型（1-加；2-减；3-乘；0-退出运算）：
1
请输入第3个矩阵的行数、列数、非零元数：
3 3 3
以三元组形式输入第3个矩阵的数据(行序，从1开始)：
1 1 1
1 3 2
3 2 1
计算结果如下：
1 1 2
6 0 0
0 1 0
A×B+C
```

连续运算

图 5: 稀疏矩阵运算器运行结果

2.5 总结

本实验中我们完成了支持多个稀疏矩阵相加、相减与相乘的运算器，经检验达到了预期效果。

未来改进方向：增加运算器功能（如增加矩阵转置、求逆等功能）。

3 源代码

本次实验所有的源码均存储在以下仓库：

<https://github.com/Hans-pokerface/Data-Structure-Project>